Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 2 : Vision embarquée à règles avec MicroPython dans OpenMV IDE

Séries de tuto sur Portenta H7, lecture 2 « Voir sans apprendre »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

31 octobre 2026

Objectifs du Tutorial

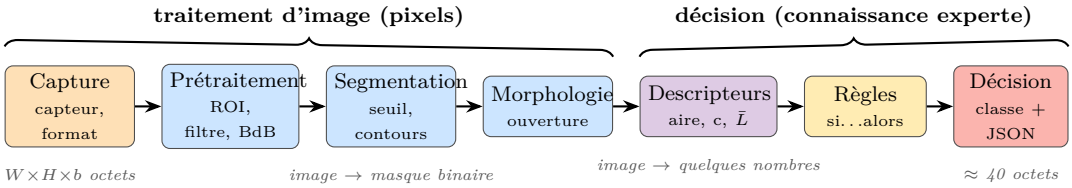
Au Tutorial 1, la Portenta a appris à **voir** : capturer une image et mesurer ce que cela coûte. Il s'agit maintenant de lui apprendre à **regarder** : trouver les objets sur le tapis, les mesurer et lire une étiquette, sans aucun réseau de neurones. Chaque étape met en œuvre un opérateur du cours et en mesure le coût.
À la fin de ce tutoriel, vous serez capable de :

- Figer l'exposition et le gain du capteur et travailler dans une région d'intérêt (ROI).
- Calculer un histogramme et un seuil automatique (Otsu), seuiller et nettoyer un masque (morphologie).
- Détecter des objets avec `find_blobs` et lire leurs descripteurs (aire, rondeur, élongation, densité).
- Trouver des contours (Canny), des droites, des cercles, des rectangles et des QR codes.
- Chronométrer chaque opérateur pour construire le budget de temps de la chaîne.

Durée indicative : 1 h 15. Ce tutoriel prépare le TP2 (détection, suivi et classification sur le tapis).

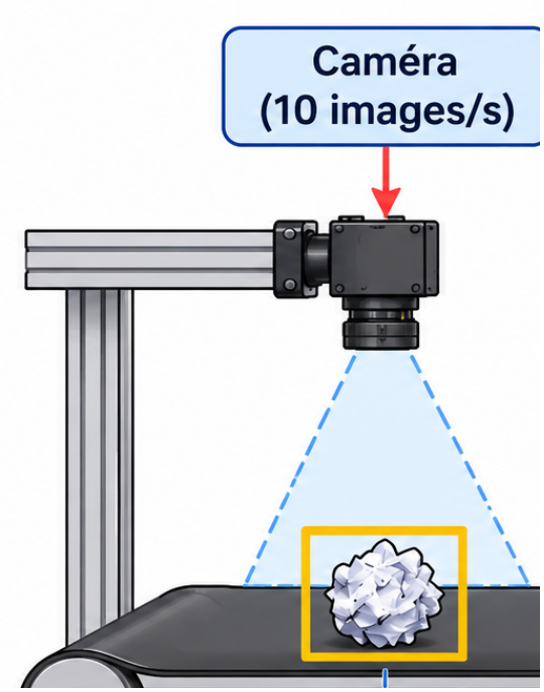
1 Introduction : la chaîne de vision à règles

Un système de vision « classique » enchaîne des opérateurs conçus à la main : on segmente l'image pour séparer les objets du fond, on décrit chaque objet par quelques nombres, puis on applique des règles. Toutes les fonctions utiles sont déjà écrites en C dans le firmware OpenMV et appelables en une ligne de MicroPython.



La chaîne de traitement que nous allons construire pas à pas. Les premières étapes manipulent tous les pixels ; les dernières ne manipulent plus que quelques nombres par objet.

Ce que nous allons traiter



Dans la station EcoTri 4.0, la Portenta observe le tapis par le dessus. Nous allons isoler les objets qui passent sous la caméra et les décrire.

Rappel : le Vision Shield est monochrome

Le capteur Himax du Vision Shield ne fournit que des **niveaux de gris**. Toutes les fonctions de ce tutoriel fonctionnent en `GRAYSCALE` : les seuils sont des intervalles d'intensité `(min, max)` entre 0 et 255. Avec une caméra couleur, les mêmes fonctions acceptent des seuils LAB à six valeurs `(Lmin, Lmax, Amin, Amax, Bmin, Bmax)` ; nous les utiliserons sur l'ESP32-CAM au TP2.

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** avec le **Vision Shield**, câble USB-C.
- Un fond uniforme et sombre (carton noir, tapis) d'environ 30×20 cm, éclairé par une lampe de bureau.
- Quelques objets : canette, bouchon ou couvercle rond, morceau de carton rectangulaire, bouteille plastique.
- Un QR code imprimé (environ 5 cm) : générez-le avec n'importe quel générateur en ligne, contenu `L-2027-0412`.

Logiciel

1. **OpenMV IDE (version 4.0 ou plus)** et la Portenta à jour (Tutorial 1).
   - Documentation des modules `sensor` et `image` : https://docs.openmv.io
2. Fixez la Portenta à 25–35 cm au-dessus du fond, objectif vers le bas (une potence ou une pile de livres suffit).

Astuce

Les versions récentes du firmware OpenMV introduisent un module `csi` qui remplace progressivement `sensor`. Les scripts ci-dessous utilisent `sensor`, toujours disponible ; si l'IDE signale une fonction obsolète, consultez la page « Migration » de la documentation.

3 Étapes détaillées du tutoriel

Étape 1 : Stabiliser la mesure (exposition, gain, ROI)

Un seuil fixe n'a de sens que si le capteur ne change pas ses réglages pendant la mesure. On laisse les automatismes converger sur le fond vide, puis on les fige.

```
import sensor, time

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)          # 320x240
sensor.skip_frames(time=2000)              # convergence AE/AGC sur le fond VIDE

try:
    g = sensor.get_gain_db()
    e = sensor.get_exposure_us()
    sensor.set_auto_gain(False, gain_db=g)
    sensor.set_auto_exposure(False, exposure_us=e)
    print("Réglages figés : gain %.1f dB, exposition %d us" % (g, e))
except Exception as err:
    print("Réglage manuel non disponible sur ce capteur :", err)

TAPIS = (0, 40, 320, 160)                  # ROI : bande utile (x, y, w, h)
clock = time.clock()
while True:
    clock.tick()
    img = sensor.snapshot()
    img.draw_rectangle(TAPIS, color=255)
    st = img.get_statistics(roi=TAPIS)
    print("moyenne %3d  écart-type %4.1f  %.1f FPS" % (st.mean(), st.stdev(), clock.
    fps()))
```

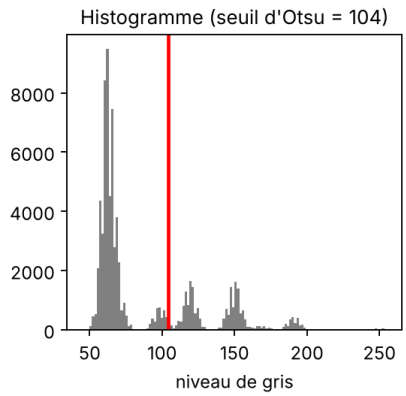
— Ajustez `TAPIS` pour qu'il couvre exactement votre fond : tout ce qui est hors de la ROI sera ignoré par les étapes suivantes.

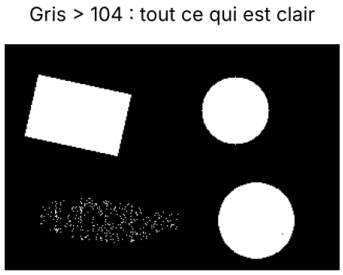
— Posez puis retirez un objet clair : avec les réglages figés, la moyenne du fond ne doit plus bouger quand l'objet entre dans le champ.

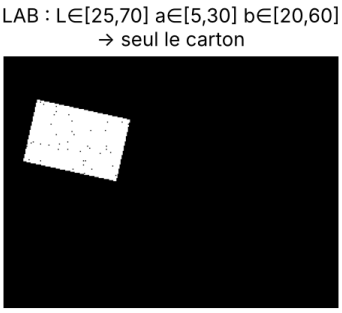
Question

Commentez la ligne `set_auto_exposure(False, ...)` et recommencez. Que devient la moyenne du fond quand un carton blanc entre dans le champ ? Pourquoi est-ce un problème pour un seuil fixe ?

Étape 2 : Histogramme et seuil d'Otsu







Rappel du cours : l'histogramme d'une scène du tapis et le seuil d'Otsu (rouge). En gris, le seuil sépare le clair du sombre, donc les objets du tapis, mais pas les matériaux entre eux.

```
import sensor, time
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QVGA)
sensor.skip_frames(time=2000)
TAPIS = (0, 40, 320, 160)

while True:
    img = sensor.snapshot()
    t0 = time.ticks_us()
    h = img.get_histogram(roi=TAPIS)
    seuil = h.get_threshold().value()       # méthode d'Otsu
    t1 = time.ticks_us()
    print("Otsu = %d  (calcul %d us)" % (seuil, time.ticks_diff(t1, t0)))
```

1. Relevez le seuil sur le fond vide, puis avec un objet clair. Pourquoi varie-t-il autant sans objet ?
2. Dans OpenMV IDE, ouvrez **Tools** → **Machine Vision** → **Threshold Editor**, choisissez la source *Frame Buffer* et déplacez les curseurs jusqu'à ce que seuls les objets apparaissent en blanc. Notez l'intervalle obtenu : c'est votre seuil `OBJET`.

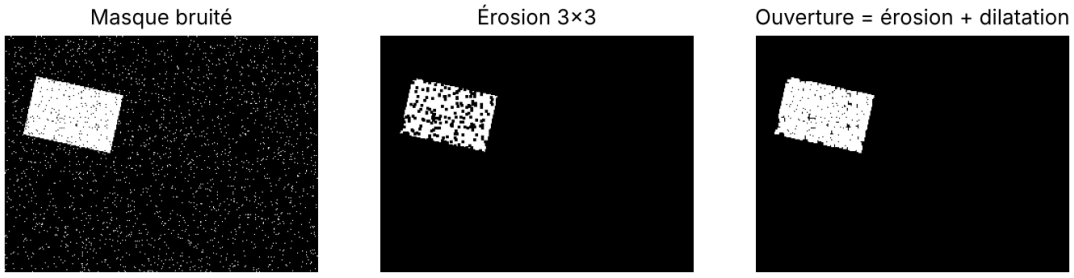
Étape 3 : Seuiller et nettoyer le masque

```
import sensor, time
```

```
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QVGA)
sensor.skip_frames(time=2000)
OBJET = [(110, 255)]                          # à remplacer par votre seuil (étape 2)

def chrono(f):
    t = time.ticks_us(); f(); return time.ticks_diff(time.ticks_us(), t)

while True:
    img = sensor.snapshot()
    tb = chrono(lambda: img.binary(OBJET))    # masque : 255 = objet
    te = chrono(lambda: img.erode(1))         # noyau 3x3
    td = chrono(lambda: img.dilate(1))        # érosion + dilatation = ouverture
    print("binaire %d us | érosion %d us | dilatation %d us" % (tb, te, td))
```



Effet attendu : l'érosion supprime les points isolés, la dilatation rend aux objets leur taille.

Question

Remplacez `erode(1)` par `erode(2)` (noyau 5×5). Comment évoluent le temps de calcul et la forme des objets ? Quel est le plus petit objet qui survit à l'ouverture ?

Étape 4 : Détecter les objets avec `find_blobs`

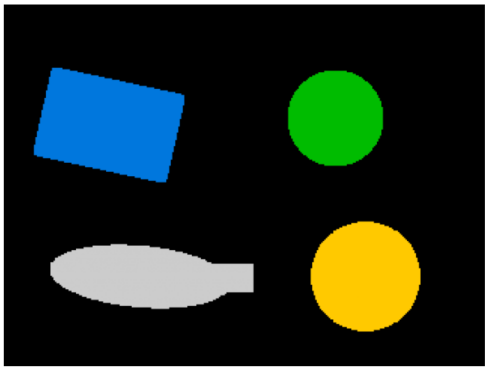
On revient à l'image en niveaux de gris (sans `binary`) : `find_blobs` seuille, étiquette les composantes connexes et calcule les descripteurs en une seule passe.

```
import sensor, time
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QVGA)
sensor.skip_frames(time=2000)
OBJET = [(110, 255)]
TAPIS = (0, 40, 320, 160)
clock = time.clock()

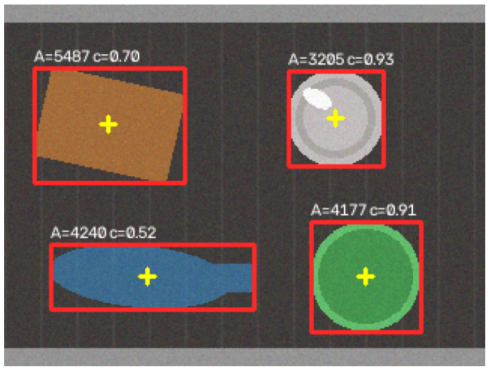
while True:
    clock.tick()
    img = sensor.snapshot()
    blobs = img.find_blobs(OBJET, roi=TAPIS, pixels_threshold=300,
                           area_threshold=300, merge=True, margin=5)
    for b in blobs:
        st = img.get_statistics(roi=b.rect())          # luminance de l'objet
        img.draw_rectangle(b.rect(), color=255)
        img.draw_cross(b.cx(), b.cy(), color=0)
        print("A=%5d rond=%.2f elong=%.2f dens=%.2f  L=%3d sd=%4.1f"
              % (b.pixels(), b.roundness(), b.elongation(), b.density(),
                 st.mean(), st.stdev()))
```

```
    print("%d objet(s), %.1f FPS" % (len(blobs), clock.fps()))
```

Étiquetage en composantes connexes



Blobs : boîte, centroïde, aire A, circularité c



À gauche, l'étiquetage en composantes connexes ; à droite, ce que vous devez obtenir : une boîte, un centroïde et des descripteurs par objet.

— `pixels()` : aire en pixels ; `roundness()` : rapport des axes de l'ellipse équivalente (1 pour un disque) ; `elongation()` : 0 pour un disque, proche de 1 pour une forme allongée ; `density()` : aire divisée par l'aire de la boîte englobante.

— `merge=True` fusionne les blobs dont les boîtes se touchent à `margin` pixels près : un objet coupé en deux par un reflet redevient un seul objet.

Objet	Aire	Rondeur	Élongation	Densité	L moyen / écart-type
Canette (vue de dessus)					
Carton					
Bouteille plastique					
Bocal ou couvercle					

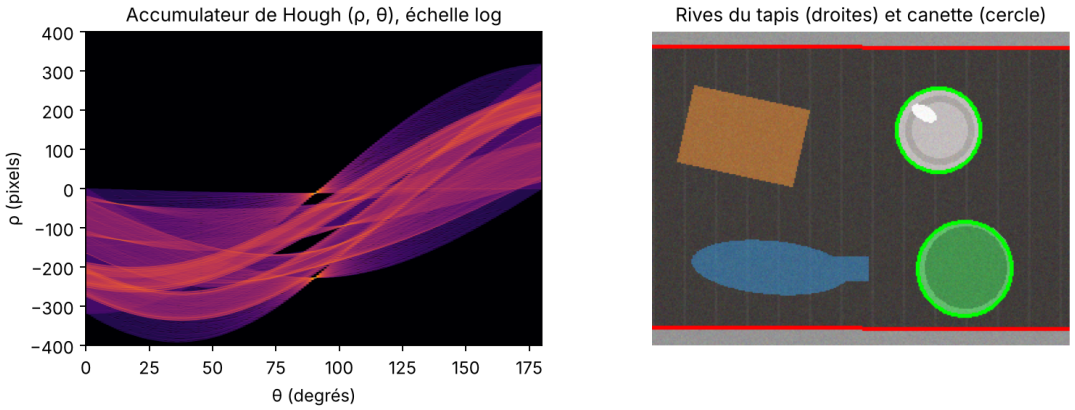
À compléter (trois mesures par objet, en le tournant) : ce tableau servira à choisir les seuils de vos règles au TP2.

Étape 5 : Contours, droites, cercles et rectangles

```
import sensor, image, time
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QQVGA)  # 160x120
sensor.skip_frames(time=2000)
MODE = 1          # 0 : Canny, 1 : cercles, 2 : droites, 3 : rectangles

while True:
    img = sensor.snapshot()
    t0 = time.ticks_us()
    if MODE == 0:
        img.find_edges(image.EDGE_CANNY, threshold=(50, 80))
        n = 0
    elif MODE == 1:
        res = img.find_circles(threshold=2000, x_margin=10, y_margin=10, r_margin=10,
```

```
                                r_min=8, r_max=30, r_step=2)
        for c in res: img.draw_circle(c.x(), c.y(), c.r(), color=255)
        n = len(res)
    elif MODE == 2:
        res = img.find_lines(threshold=1000, theta_margin=25, rho_margin=25)
        for l in res: img.draw_line(l.line(), color=255)
        n = len(res)
    else:
        res = img.find_rects(threshold=10000)
        for r in res: img.draw_rectangle(r.rect(), color=255)
        n = len(res)
    print("mode %d : %d forme(s) en %d ms" % (MODE, n, time.ticks_diff(time.ticks_us
    (), t0) // 1000))
```



Transformée de Hough : chaque pixel de contour vote dans l'accumulateur (à gauche) ; les maxima donnent les droites et les cercles (à droite).

1. Testez les quatre modes. Les recherches de formes sont coûteuses : nous travaillons en QQVGA (160×120).
2. Pour les cercles, adaptez `r_min` et `r_max` au rayon apparent de votre canette. Mesurez le temps de calcul avec `r_step=2` puis `r_step=4`.
3. Augmentez `threshold` jusqu'à ce que les fausses détections disparaissent. Que se passe-t-il pour un cercle partiellement caché ?

Question

Repassez en QVGA avec le mode cercles. Le temps de calcul est-il multiplié par 4 (le nombre de pixels) ? Reliez votre réponse au coût de la transformée de Hough vu en cours.

Étape 6 : Lire un QR code

```
import sensor, time
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE); sensor.set_framesize(sensor.
    QQVGA)
sensor.skip_frames(time=2000)
try:
    sensor.set_auto_gain(False)           # évite le « pompage » sur le blanc de l'é
    tiquette
except Exception:
```

```
    pass
clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()
    img.lens_corr(strength=1.8, zoom=1.0)   # corrige la distorsion de l'objectif
    for code in img.find_qrcodes():
        img.draw_rectangle(code.rect(), color=255)
        print("QR : %s (version %d, %d px de côté)"
              % (code.payload(), code.version(), code.w()))
    print("%.1f FPS" % clock.fps())
```

— Éloignez progressivement l'étiquette : notez la largeur `code.w()` à laquelle le décodage échoue. Divisez-la par le nombre de modules (21 pour la version 1, 29 pour la version 3) : vous retrouvez la règle des 3 à 4 pixels par module du cours.

— Inclinez l'étiquette : jusqu'à quel angle le décodage tient-il ?

Étape 7 : Budget de temps de la chaîne

Assemblez les mesures précédentes pour une image QVGA et une ROI de la taille du tapis :

Opérateur	**Temps mesuré (ms)**	**Part de l'échéance de 600 ms**
Capture `snapshot()` (Tutorial 1)		
Histogramme + Otsu		
`find_blobs` + statistiques		
`find_circles` (QQVGA)		
`find_qrcodes`		
Total		

Question

Quelle étape domine ? Est-il raisonnable de chercher les cercles et les QR codes à chaque image, ou vaut-il mieux ne les chercher que dans la boîte d'un blob déjà détecté ? Proposez l'ordre des opérations le plus économique.

4 Pour aller plus loin

— Exemples OpenMV : **File** → **Examples** → **Color Tracking**, **Feature Detection** et **Barcodes**.

— Documentation OpenMV, module `image` (`find_blobs`, `find_circles`, `find_qrcodes`) : docs.openmv.io.

— R. Szeliski (University of Washington), *Computer Vision : Algorithms and Applications*, 2e éd., chapitres 3 et 7 : szeliski.org/Book.

— Stanford CS131, *Computer Vision : Foundations and Applications*, cours sur les contours et la transformée de Hough.